

Today's Lab — Four Problems on Means-End Analysis & TSP

Program 1

Robot Traversal Using Means-End Analysis

Solve a robot traversal problem on a 4x4 grid using Means-End Analysis. Identify the difference between the current and goal state, then apply the operator (Up/Down/Left/Right) that reduces it.

Program 2

Block-World Problem Using Means-End Analysis

Use MEA to solve the classic Block-World problem. The robot must rearrange three blocks from initial state to goal stack using pickup, putdown, stack, and unstack operators.

Program 3

Travelling Salesman Problem Using Brute Force

Solve the TSP for 4 cities by enumerating all unique tours that start and end at city A. Print every tour with its cost and report the minimum-cost Hamiltonian cycle.

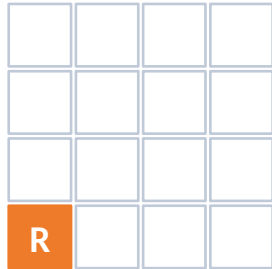
Program 4

TSP — Brute Force vs Nearest Neighbour

Run Brute Force and the Nearest-Neighbour heuristic on the same 4-city graph. Compare paths and costs; observe that the greedy NN can miss the optimal tour.

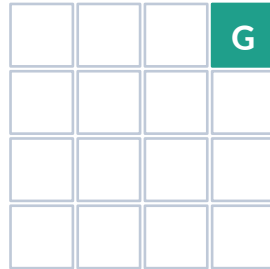
The Robot Grid You Will Use for Program 1

Initial State



R at (0,0)

Goal State



G at (3,3)

Operators and Difference

Operators: MoveUp, MoveDown, MoveLeft, MoveRight.

State: (x, y) coordinates of the robot.

Difference: $dx = x_{goal} - x_{curr}$, $dy = y_{goal} - y_{curr}$.
Reduce $|dx|$ or $|dy|$ using the matching operator (means).

For initial (0,0) -> goal (3,3):

- $dx = 3$ (apply MoveRight three times)

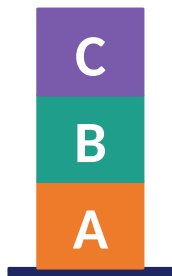
- $dy = 3$ (apply MoveUp three times)

-> **Minimum moves = 6**

The Block-World You Will Use for Program 2

Initial State

Goal State



Operators and MEA Approach

Operators: pickup(x), putdown(x),
stack(x, y), unstack(x, y).

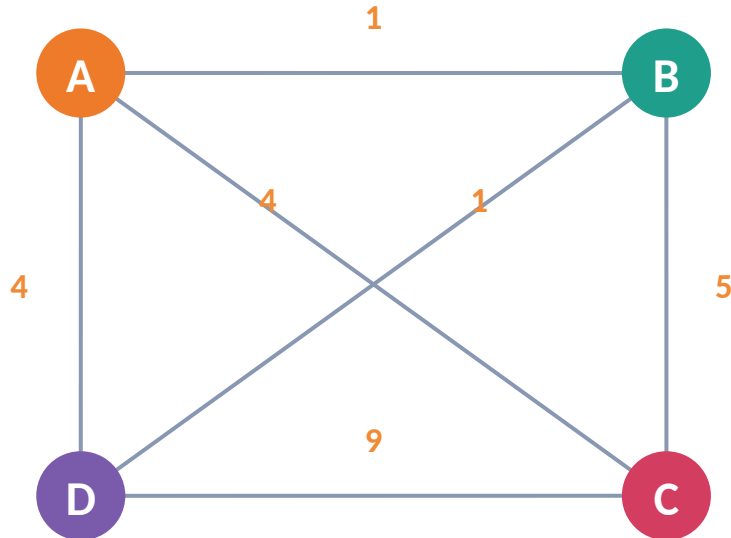
Predicates: ON(x, y), ONTABLE(x), CLEAR(x), HOLDING(x).

Goal predicates:

- ONTABLE(A), ON(B, A), ON(C, B)

MEA picks the operator whose effect reduces the difference between current and goal predicates; missing preconditions become recursive sub-goals.

The City Graph You Will Use for Programs 3 & 4 (TSP)



Numbers on edges = distance between cities. Tour must start & end at A.

TSP Statement

Visit every city exactly once and return to the start, with MINIMUM total distance.

Distance Matrix:

	A	B	C	D
A	0	1	4	4
B	1	0	5	1
C	4	5	0	9
D	4	1	9	0

Recall — Brute Force checks 3 unique tours.

Optimal cost = 14.

Program 1 — Robot Traversal Using Means-End Analysis

Program 1

Task: Use Means-End Analysis to move the robot from (0,0) to (3,3) on a 4x4 grid. Print the operator chosen at each step and the resulting state until the goal is reached.

Expected Output:

```
Initial: (0,0)  Goal: (3,3)  diff: 3,3
Step 1: MoveRight -> (1,0)  diff: 2,3
Step 2: MoveRight -> (2,0)  diff: 1,3
Step 3: MoveRight -> (3,0)  diff: 0,3
Step 4: MoveUp    -> (3,1)  diff: 0,2
Step 5: MoveUp    -> (3,2)  diff: 0,1
Step 6: MoveUp    -> (3,3)  diff: 0,0  GOAL
Total moves: 6
```

Steps to implement:

- 1 Define state as a tuple (x, y) of robot's position
- 2 Compute difference (dx, dy) between current and goal
- 3 Pick the operator that reduces |dx| or |dy|
- 4 Apply the operator and update the state
- 5 Repeat until (dx, dy) = (0, 0); print every move

Program 2 — Block-World Using Means-End Analysis

Program 2

Task: Use MEA to rearrange three blocks from the initial configuration (A, B, C separately on the table) to the goal stack (A on table, B on A, C on B). Print the operator sequence.

Expected Output:

```
Initial: ONTABLE(A), ONTABLE(B),  
         ONTABLE(C), CLEAR(A,B,C)  
Goal:    ONTABLE(A), ON(B,A), ON(C,B)  
Step 1: pickup(B)  
Step 2: stack(B, A)    -> ON(B, A)  
Step 3: pickup(C)  
Step 4: stack(C, B)    -> ON(C, B)  GOAL  
Total moves: 4
```

Steps to implement:

- 1 Represent the state as a set of predicates
- 2 Compute the difference vs goal predicates
- 3 Pick an operator whose effect reduces the difference
- 4 Recursively achieve any unmet preconditions
- 5 Apply, update state, repeat until goal holds

Program 3 — TSP Using Brute Force

Program 3

Task: Solve TSP on the 4-city graph by enumerating every unique tour that starts and ends at A. Print every tour with its cost and the optimal Hamiltonian cycle.

Expected Output:

```
Tours from A (3 unique cycles):  
A -> B -> C -> D -> A : 1+5+9+4 = 19  
A -> B -> D -> C -> A : 1+1+9+4 = 15  
A -> C -> B -> D -> A : 4+5+1+4 = 14  <- OPTIMAL  
Optimal tour: A -> C -> B -> D -> A  
Optimal cost: 14
```

Steps to implement:

- 1 Build distance matrix from the city graph
- 2 Generate all permutations of cities except A
- 3 For each permutation compute cycle cost $A \rightarrow \dots \rightarrow A$
- 4 Track the minimum-cost tour and its path
- 5 Print every tour and the optimal cost

Program 4 — TSP: Brute Force vs Nearest Neighbour

Program 4

Task: Run Brute Force AND Nearest Neighbour on the same 4-city graph. Print both tours and their costs side-by-side; observe that the greedy NN can MISS the optimal tour.

Expected Output:

Brute Force:

Tour : A -> C -> B -> D -> A

Cost : 14 (OPTIMAL)

Nearest Neighbour (start A):

A -> B (1) -> D (1) -> C (9) -> A (4)

Cost : 1+1+9+4 = 15 (sub-optimal)

NN error vs optimal = +7.1%

Steps to implement:

- 1 Reuse the Brute Force solver from Program 3
- 2 Implement Nearest Neighbour starting at A
- 3 Always move to the closest unvisited city
- 4 Return to A once all cities are visited
- 5 Print both tours, costs and the relative error